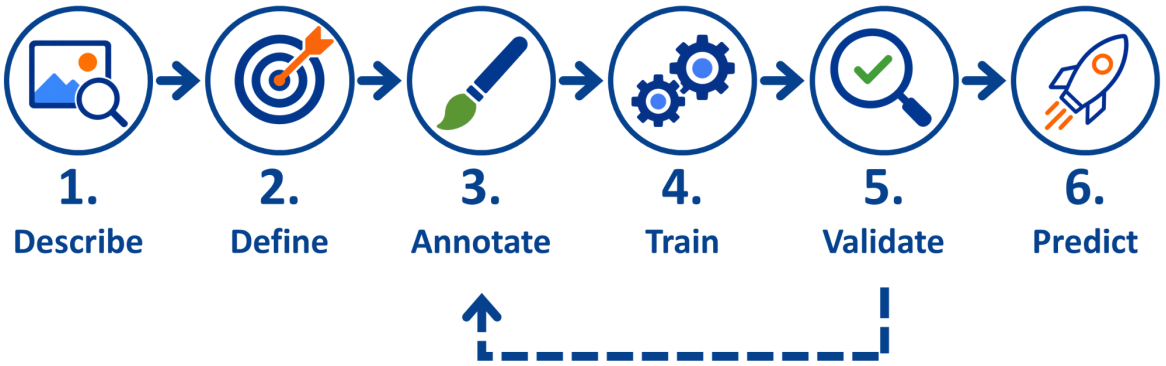
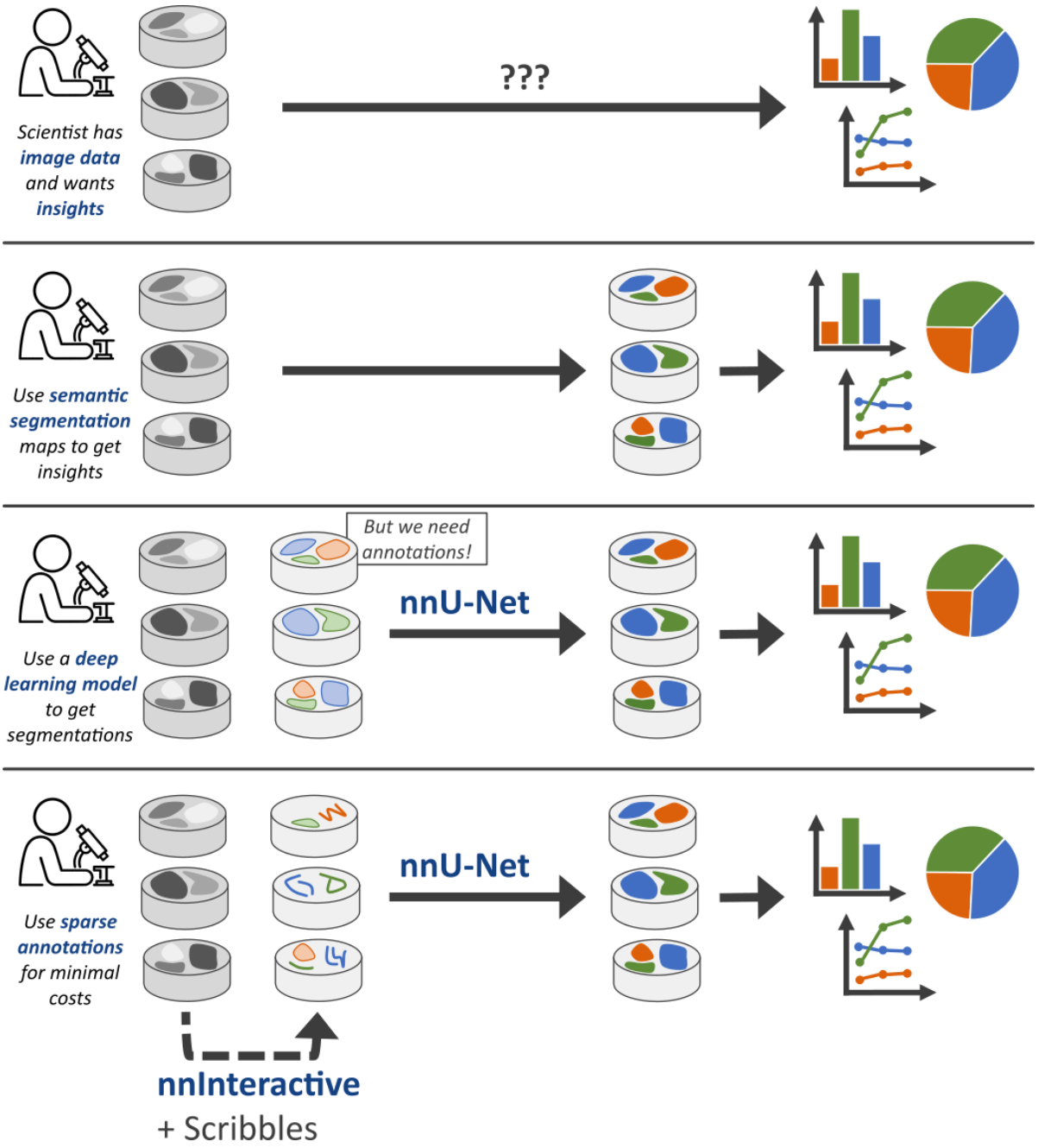
Annotation Workflow



Summary: *Scientific images contain valuable quantitative information, but extracting it often requires segmentation of the structures of interest. This workflow provides a practical way to turn expert knowledge into automatic image segmentations while keeping annotation effort as low as possible. The process starts by organizing and understanding the data, defining clear classes, and creating representative annotations. Annotation effort is minimized by combining* ***nnInteractive*** *and* ***scribble annotations****. These annotations are then used to train* ***nnU-Net****, a self-configuring method for automatic image segmentation. The workflow is iterative: predictions are reviewed, failure cases are identified, and additional annotations are added where they are most useful. Once the results are sufficient, the trained model can be applied to new images to generate segmentations automatically for downstream scientific analysis.*

Motivation

Turn scientific images into quantitative insights.



Setup

Organize your data for a smooth and structured workflow.

We use the standard nnU-Net folder structure to stay compatible with the existing nnU-Net workflow. This keeps the overall workflow simple, avoids redundant conversions or file movements, and helps to keep the project clear and reproducible. We need three main storage folders:

- **nnUNet_raw/:** This folder contains the original datasets in the format expected by nnU-Net. Also, the generated annotations will be stored here.
- **nnUNet_preprocessed/:** This folder is filled automatically when running nnU-Net planning and preprocessing and is used during model training. You don't have to do anything here manually.

- **nnUNet_results/:** This folder stores trained models and training outputs.

To start, create a folder in nnUNet_raw and follow the naming conversion DatasetXXX_DatasetName (XXX is a unique three-digit dataset ID), for example, Dataset042_CellSegmentation. Your folder structure should look like this and should contain the following subfolders.

```
nnUNet_raw/
└── Dataset042_CellSegmentation/
      ├── imagesTr/              # Store the images you annotated
here
      │     ├── Case001_0000.nii.gz # Images need a _0000 postfix
      │     └── ...
      ├── annotationsTr/         # Store your manual annotations
here
      │     ├── Case001.nii.gz
      │     └── ...
      └── labelsTr/              # this will be generated from your
            ├── Case001.nii.gz    # annotationsTr automatically
            └── ...
```

The format that works best for manual annotation differs slightly from the format nnU-Net requires for training. Therefore, human-created annotations are stored in annotationsTr and converted before training. The converted labels are saved in labelsTr, which is the folder nnU-Net uses during training**.**

Note: In the end**,** imagesTr, annotationsTr and labelsTr must contain the same number of files. Each image,its corresponding annotation and label must also have the same filename. Images need a _0000 postfix.

Note: We recommend storing images and annotations as NIfTI files (.nii.gz). This format is supported by nnU-Net and by most nnInteractive integrations. But in general, other formats are also possible.

---

1. Describe

Understand the full range of your image data and the structures you are interested in.

Answer and document the following questions:

Images

- What type of Images do I have? e.g. soil-core CT or bone marrow Microscopy.
- How many different Images are available?
- How much do the images vary in contrast, brightness, noise, resolution, artifacts, background, and acquisition conditions?
- What's a representative image?
- What is the most difficult image I want the model to work on?

Target Structures

- Which structures am I interested in?
- How can I separate these structures into distinguishable classes?
- How much do individual objects of a class vary in size, shape, appearance, orientation,

position, and context?
- How often does each class occur, and in how many images or volumes is it represented? Are some classes much more common or rarer than others?

---

2. Define

Define your task precisely and resolve all ambiguities.

Class Definitions

Define all classes required for the segmentation task and assign each class a unique ID. Use continuous numbering starting at 0. The first two IDs are reserved.

0: Ignore
1: Background
2: *<First class>*
3: *<Second class>*
4: …

Note: *Ignore refers to all voxels that are not labeled and for which we provide no information to the model.*
Note: *Background is a class and must be explicitly defined and labeled.*

Class Descriptions

For each Class document:
Description: What is the structure?
Visual Description: What do objects in this class look like? Define what should be included or excluded, where the boundaries are, and how uncertain or unusual cases should be handled.
Example images: Add representative examples.

---

3. Annotate

Minimize the annotation effort while maximizing model performance.

3.1. Annotation Principles

Since we want to minimize the annotation effort, the labels we generate should contain as much information as possible and should have the highest possible quality. To do so, follow these principles:

Diversity: To help the model generalize well for your data, capture the natural variability in your dataset, such as:

- **Image-level variability**: differences in contrast, brightness, noise, resolution, artifacts, background, and acquisition conditions.
- **Class-level variability**: differences in object size, shape, appearance, orientation, position, and context.

Consistency: Apply the same annotation criteria throughout the entire dataset. Similar structures or objects should always be labeled in the same way to provide the model with reliable training examples. The better you define your classes in stage 2, the easier it is to label them consistently.

Balance: Ensure that all classes are represented by a sufficient and reasonably balanced number of annotations whenever possible. This helps prevent the model from becoming biased toward more frequently annotated classes.

Note: A dense 3D mask (from e.g. nnInteractive) contains much more information than a sparse

scribble annotation. Therefore, many more scribble annotations are typically required to achieve comparable coverage.

3.2. Annotation Instruction

3.2.1. Image Selection

To capture the **diversity** of the dataset, distribute annotations across as many different images as possible. Annotating small regions in several images is usually more valuable than annotating one large region in a single image. Random image selection is often a good starting point. If some classes, image types, or difficult cases are underrepresented, manually select additional images to correct the **imbalance**.

3.2.2. Region Selection

Loosely **cluster annotations** within each image. Whenever possible, annotate nearby objects from different classes so the model can learn how to distinguish between them. Two classes next to each other provide more information than an isolated class. Also, here random region selection is often a good starting point. For example, select a random slice index and annotate structures in and around that slice. Again, if some classes, regions, or difficult cases are underrepresented, manually select additional areas to correct the **imbalance**.

3.2.3. Class Selection

In most datasets, there is **no natural class balance**, meaning that some classes occur much more frequently than others. Try to roughly balance the classes you annotate. You do not need to count voxels. If some classes are very rare, manually select regions containing these classes. When doing so, also annotate the surrounding classes so the model can learn to distinguish them. Ideally, each annotated image should contain examples of all classes. There are also situations where an intentional imbalance is beneficial, for example if some classes are much more complex or variable than others and therefore require more training examples.

3.2.4. Tool Selection

Use **nnInteractive** to generate **3D annotations** whenever possible. nnInteractive is suitable when all of the following conditions are met:

- Your data is 3D.
- The target class consists of **individual, separable objects** (e.g., individual cells) since nnInteractive is doing Instance Segmentation.
- The objects are **not excessively large**; otherwise, the runtime of nnInteractive will increase drastically and is not feasible anymore.
- nnInteractive is able to produce **meaningful segmentations** for the objects in this class (trial and error)

If nnInteractive cannot be applied or does not produce satisfactory results, use **scribble annotations** as a fallback. Typically this is the case for at least the background class.

Napari and MITK support both nnInteractive and scribble annotations using the brush tool. You can therefore combine both methods freely and dynamically within the same annotation mask.

3.2.5. nnInteractive

nnInteractive: An interactive annotation tool which can generate 3D instance segmentation masks from simple 2D user input like clicks, boxes, lasso or scribbles.
Links: Paper | Code | Napari Integration | MITK Integration

nnInteractive creates instance segmentations, meaning that individual objects are segmented one by one. Since we later train a semantic segmentation model, the individual object masks are combined into a semantic map in which all objects belonging to the same class share the same class ID.

- **Enable the semantic map output** in the **Label Aggregation** box.
- **Select the image** you want to annotate.
- **Initialize nnInteractive**.
- For each object you want to segment:
    - Check the class ID in **Label Aggregation** and set it to the correct class before starting.
    - Select an interaction type, such as a point, box, scribble, or lasso, and place it in the image.
    - Add further interactions until the segmentation is satisfactory.
    - When nnInteractive produces only a partly correct result, switch to the brush tool and manually correct the segmentation. Do this only as a last step and don't add interactions to this object afterwards.
    - Click **Next Object** and repeat the process.
- You can also add scribbles directly to the output segmentation layer while working. See the scribble annotation instructions below.
- Save the output layer in the *annotationsTr* folder using the same filename as the corresponding image, but without the *_0000* postfix.

References

Isensee, F.*, Rokuss, M.*, Krämer, L.*, Dinkelacker, S., Ravindran, A., Stritzke, F., Hamm, B., Wald, T., Langenberg, M., Ulrich, C., Deissler, J., Floca, R., & Maier-Hein, K. (2025). nnInteractive: Redefining 3D Promptable Segmentation.

*: equal contribution

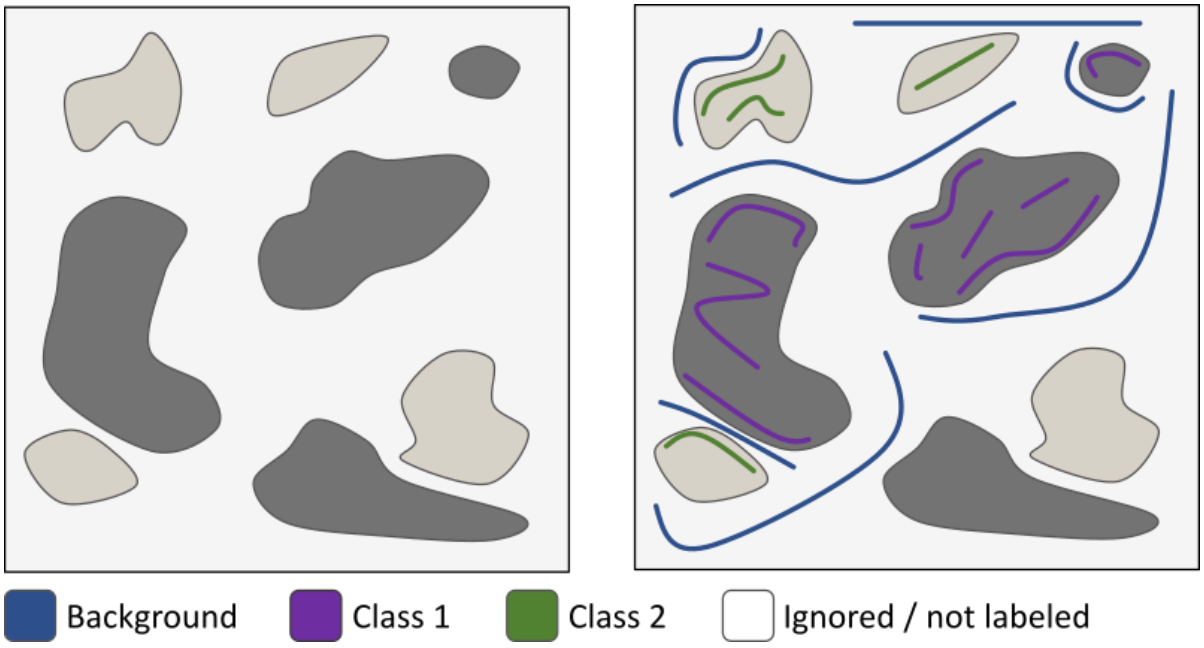
3.2.6. Scribble Annotations

Scribble Annotations: Scribble annotations are lines/scribbles drawn directly on clearly identifiable structures in an image, typically with some kind of brush tool. Instead of marking the complete structure, you label only a few representative areas, while everything else remains unlabeled. Besides the general annotation principles and selection strategies, follow these guidelines for effective scribble annotations:

- **Label all classes.** Provide enough representative examples for the model to learn how the classes differ from one another. Depending on their diversity and complexity, some classes may require more annotations than others.
- **Keep scribbles inside the target structure.** Scribbles that cross into another class can confuse the model.
- **Annotate near class boundaries.** Place scribbles close to borders so the model can learn where one class ends and another begins. However, do not focus only on boundaries, also include representative areas within each class.

- **Label neighboring classes together.** When two classes touch, place scribbles on both sides of the boundary.
- **Be efficient.** Scribbles do not need pixel-perfect precision. A few well-placed strokes usually provide the best balance between annotation effort and segmentation quality.
- **Don’t forget the Background.** This is also a class we have to label, so don't forget to add enough background annotations. Otherwise, the model may not learn boundaries of other classes accurately, resulting in fuzzy or imprecise borders.

Example Scribble Annotations:



3.2.7. Q&A

How much data do I have to label?
There is no general answer to this question. The number of annotations needed for each class depends on the diversity of the data and the complexity of the segmentation task. In general, more representative training examples improve the model, so when in doubt, add a few more.

How many iterations do I need?
Stop when you are satisfied with the results. When additional annotations produce only minor improvements, the model may be approaching its current performance limit. Achieving better results could then require a much larger annotation effort, so consider whether the expected gain is worth the investment. Before stopping, make sure the plateau is not caused by systematic annotation errors, inconsistent class definitions, or underrepresented failure cases.

How to balance the number of annotations and the number of iterations?
If annotation time is limited, it can be useful to work with smaller annotation batches and retrain more often. This gives you more frequent feedback and helps you focus new annotations on the cases where the model still struggles. If compute time is more limited, it may be better to collect a larger set of annotations before retraining and therefore use fewer iterations.

Can I use some other annotation tools?
Yes. Other annotation tools can also be used, and for some types of data or structures they may even be more suitable. However, use them carefully and make sure they do not introduce systematic biases, for example by favoring certain object sizes, shapes, or easier cases. The annotations should

remain representative, consistent, and follow the same class definitions and labeling rules throughout the dataset.

---

4. Train

Turn your annotations into an automated segmentation model.

---

nnU-Net: An automated segmentation framework that configures, plans, trains, and applies deep learning models with minimal manual setup.
Links: [Paper] | [Code]

Preprocessing:

1. **Rename:** nnUNet expects a "_0000" extension to your image files; if it has not already happened, add the extension my_file.nii.gz → my_file_0000.nii.gz. This is only needed for the images, not the labels or annotations.
   ```
   imageTr:          my_file_0000.nii.gz
   annotationsTr:    my_file.nii.gz
   labelsTr:         my_file.nii.gz
   ```
2. **Shift labels:** Most image viewers display label 0 as transparent, making it the most convenient choice for the Ignore label during annotation. However, nnU-Net expects the Ignore label to be the highest label ID. Therefore, before training, we need to shift all class labels by −1 and move the Ignore label to the last label index. The result needs to be stored in your labelsTr folder.
3. **Create dataset.json:** Add the dataset.json file to your dataset folder; some documentation can be found here. An example could look like this:
   ```
   {
     "channel_names": {
       "0": "MRT"
     },
     "labels": {
       "background": 0,
       "class_1": 1,
       "class_2": 2,
       "ignore": 3
     },
     "numTraining": 20,
     "file_ending": ".nii.gz"
   }
   ```
4. **nnU-Net preprocessing:** Run nnUNet plan and preprocessing to generate the dataset fingerprint, create the preprocessing plan, and preprocess the images for training.
   ```
   nnUNetv2_plan_and_preprocess -d DATASET_ID --verify_dataset_integrity
   ```

Training

- **Train the model:**
  ```
  nnUNetv2_train DATASET_ID CONFIGURATION FOLD
  ```
  - For example, train fold 0 of a 3D full-resolution model:
    ```
    nnUNetv2_train 42 3d_fullres 0
    ```

- **Cross Validation:** Train for all folds separately:
  ```
  nnUNetv2_train 42 3d_fullres 0
  nnUNetv2_train 42 3d_fullres 1
  nnUNetv2_train 42 3d_fullres 2
  nnUNetv2_train 42 3d_fullres 3
  nnUNetv2_train 42 3d_fullres 4
  ```
- **Single model:** Use all available training data for a single model:
  ```
  nnUNetv2_train 42 3d_fullres all
  ```

References

Isensee, F., Jaeger, P. F., Kohl, S. A., Petersen, J., & Maier-Hein, K. H. (2021). nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. Nature Methods, 18(2), 203-211.

---

5. Validate

Decide if your model is ready or not.

After training finishes, nnU-Net predicts every image assigned to the validation fold. Review these predictions carefully to decide whether the model quality is sufficient or another iteration of annotations is needed.

```
nnUNet_results/
└── DatasetXXX_Name/
      └── nnUNetTrainer__nnUNetPlans__CONFIGURATION/
            ├── fold_0/
            │     └── validation/
            │           ├── <case_1>
            │           ├── <case_2>
            │           ├── ...
            │           └── summary.json
            ├── fold_1/
            ├── fold_2/
            ├── fold_3/
            └── fold_4/
```

The *summary.json* file contains validation metrics for each case and class, together with average values across the validation set. These values are useful for tracking progress and comparing training iterations. However, when sparse annotations are used, the metrics should be treated mainly as an indicator.
When using cross-validation, repeat this review for all folds. Each image is used for validation in one fold, allowing you to inspect predictions across the complete training dataset. During later inference, nnU-Net uses the models from all trained folds as an ensemble by default. Averaging their predictions often produces more robust and accurate results than using a single fold model. The final

inference quality may therefore be slightly better than the predictions found in an individual fold's *validation* folder.

Based on the validation results, either return to Stage 3 for another annotation cycle or continue to Stage 6.

---

6. Predict

Release the model and use it in the wild.

To predict new or autocomplete training images, place the images in an input folder using the same file format and naming convention as the training images, including channel postfixes such as _0000. Then run:

```
nnUNetv2_predict -i INPUT_FOLDER -o OUTPUT_FOLDER -d DATASET_ID -c
CONFIGURATION
```

Example:

```
nnUNetv2_predict -i imagesTs/ -o predictions/ -d 42 -c 3d_fullres
```

By default, nnU-Net combines the five cross-validation folds during prediction. All five folds must therefore be trained. When using a model trained on all images with fold_all, specify:

```
nnUNetv2_predict -i imagesTs/ -o predictions/ -d 42 -c 3d_fullres -f all
```

The output folder contains one segmentation file for each input case. Review the predictions before using them for further analysis.